

EEE 475/575 Medical Image Reconstruction & Processing
Homework #5
6 May 2026, Friday at 23:59

GUIDELINES FOR HOMEWORK SUBMISSION

- Upload your solutions to Moodle as **two separate files**:
 - 1) **PDF Report File:** The report should be typeset (i.e., no handwriting) and should properly present your results. In addition, the report should display the relevant sections of your code at the end of each subpart. No partial credits will be given to answers with missing codes or unjustified answers.

At the beginning of your report, provide a section titled “GenAI Usage Disclosure” and clearly explain whether you have used any genAI tools. Include the name/version of the tool and a brief description of how it was used (see the “GenAI Policy” included below).
 - 2) **Code File:** You can use any programming platform of your choice. If you do not upload your code file, your homework will not be evaluated (i.e., 0 pts).
- Submission system will remain open for 1 day after the deadline:
 - No points will be lost if you submit your assignment within 12 hours past the deadline.
 - There will be a 50% penalty if you submit after 12 hours but within 24 hours past the deadline.
 - No submissions beyond 24 hours past the deadline.

GenAI Policy: Generative AI (genAI) tools (e.g., large language models, coding assistants) may be used for homework assignments and project to assist with coding and to improve writing. Any use of genAI must be clearly disclosed, including the name/version of the tool and a brief description of how it was used. Upon request, the student should be ready to provide prompts, outputs, and execution logs pertaining to their genAI use for all assignments. Students remain fully responsible for all submitted material and must be able to explain and justify their code, results, and written content. Reliance on genAI does not transfer responsibility for correctness or originality. Failure to disclose AI usage or inappropriate reliance on AI-generated content will be treated as a violation of academic integrity.

PREPARATION

From the Moodle page of the class, download the following files:

- **multicoil-random.mat:** Contains fully-sampled 2DFT data acquired using an 8-channel coil, together with random sampling masks. `im(Nx,Ny,Nc)` contains the fully-sampled images for each channel. `mask(Nx,Ny)` contains a random sampling mask with 32×32 central calibration region, showing which k-space locations will be sampled. Here, $N_x = 256$ and $N_y = 256$ are image dimensions, and $N_c = 8$ is the number of coils.

Side Note: If you display the k-space data corresponding to `im`, you will see that some k-space locations have values very close to zero. This is because the actual k-space data size was 220×160 . Because the wavelet operator (see below) requires image sizes that are powers of 2, the original data has been padded with zeros to 256×256 in k-space. Ignore this fact for the purposes of this homework.

- **@Wavelet.zip:** This file contains @Wavelet folder in it. Keep this folder in your working directory. When needed in this homework, generate a wavelet operator, perform forward and inverse wavelet transforms as follows:

```
>> wv = Wavelet('Daubechies',4,4); % generate wavelet operator
>> coeffW = wv*im; % take forward wavelet transform
>> im2 = wv'*coeffW; % take inverse wavelet transform. Note that we use the
adjoint wv (i.e., wv'), which is essentially the inverse operator.
```

Note 1: Throughout this homework, use the “montage” function whenever you need to display images from all coils in a compact form.

Note 2: PSNR and SSIM are very sensitive to normalization. For ALL final reconstructed images (i.e., all of the SoS combined images in this homework), firstly normalize the images by their 95th percentile pixel intensities. You can use “*prctile*” built-in function of MATLAB to compute the 95th percentile pixel intensity for a given image *im* as follows: `prctile(im(:),95)`.

Homework #5

This homework is on the implementation of l_1 -SPIRiT **reconstruction** for compressed sensing reconstruction of randomly undersampled Cartesian MRI data sets.

1) Display the Data: Display the magnitude images for all coils, and the corresponding k-space spectrums. Perform a sum-of-squares (SoS) combination of coil images and display the final image. We will use this image as the reference image in the rest of the homework. Define a variable named *max_val* as the maximum pixel intensity of this image (after 95th percentile normalization).

Important Note: During PSNR and SSIM computations, normalize both the image of interest and the reference image by *max_val*.

2) Sampling Mask and Random Undersampled Images: Display the sampling mask as an image. This mask is a “logical” matrix made of ones and zeros, where ones correspond to the locations that will be sampled during MRI acquisition. Determine the overall acceleration corresponding to the given mask, i.e., $R = 1/(\text{ratio of k-space acquired})$.

Write a function `[imu, Mu] = randundersample(im, mask)` that uses the mask to generate the random undersampled k-space data and the corresponding images. Both “imu” and “Mu” are matrices of size $N_x \times N_y \times N_c$. To do this, first take centered 2D FFT of the image for each coil to compute the corresponding k-space data. Then, a simple element-wise multiplication of the mask with the k-space data is sufficient to generate the undersampled k-space data for that coil. Finally, imu is the zero-fill reconstructed images for all coils (i.e., centered inverse 2D FFT of Mu for each coil).

Display the resulting k-space spectrums, and the magnitude of the zero-fill images for all coils using the “montage” function. Perform a SoS combination of coil images and display it using *imshow* grayscale window of `[0 max_val]`. Display the error images using *imshow* grayscale window of `[0 0.2*max_val]`. Compute PSNR and SSIM. Briefly comment on how these undersampled images look when compared to uniformly undersampled images (i.e., skipping lines in k-space) that you saw in Homework #4.

3) l_1 Regularization in Wavelet Domain: Write a function that takes an image and returns a version that is l_1 regularized in wavelet domain (i.e., sparsity is enforced in wavelet domain): `imth = llwavelet(imr,beta)`, where *imr* is an input 2D image and β is the relative threshold in wavelet domain. This function will compute the wavelet transform of the image *imr*, apply the following soft thresholding on each wavelet coefficient, and take inverse wavelet transform:

$$S_{\beta}(x) = \frac{x}{|x|} \max(0, |x| - \beta|x|_{\max})$$

Here, $|x|_{\max}$ is the maximum wavelet coefficient magnitude, and $\beta|x|_{\max}$ is the threshold in wavelet domain. Hence, β is defined relative to the maximum coefficient magnitude. As an example, choosing $\beta=10^{-3}$ will set the coefficients that are smaller than 1/1000 of the maximum coefficient to zero.

Apply wavelet regularization on the images from Part (2) (i.e., random undersampled images from each coil). Note that you will apply wavelet regularization on complex-valued images, and not magnitude images.

First, display the magnitude of the wavelet coefficients for all coils, before thresholding (**Note:** Display them in logarithmic scale, like we do for k-space spectrum). Next, do the following for 3 different β values (choose the values so that you can clearly see the effect of β): Perform a SoS combination of coil images and display the final image using a window of `[0 max_val]`. Display the error images using a window of `[0 0.2*max_val]`. Compute PSNR and SSIM. Briefly comment on the results.

4) SPIRiT Kernel Weights: In this homework, we will only consider the case for a 3×3 kernel for simplicity. Write a function that takes the central 32×32 calibration data in k-space to compute SPIRiT kernel weights, `kernel = calibrateSpirit(Mc, lambda)`. Here, M_c is the central 32×32 k-space data extracted from the calibration region, λ is the regularization parameter used in kernel estimation stage of SPIRiT.

Note that for a 3×3 kernel, the calibration utilizes all 8 acquired neighborhood samples for the chosen coil, and all 9 acquired samples from the other coils. Therefore, “kernel” is a 2D matrix of size $(9 \times N_c - 1) \times N_c$. Display the magnitude of the kernel as an image for $\lambda = 0$. Note that because SPIRiT kernel is sample geometry independent, this kernel applies for all acceleration levels, and for all k-space locations.

5) SPIRiT Reconstruction: Write a function `[imr, Mr] = spirit(Mu, kernel)` that takes randomly undersampled k-space data M_u and the SPIRiT kernel as inputs. It then performs one iteration of SPIRiT reconstruction (i.e., applies the kernel) to return the resulting k-space data and the corresponding coil images. Both “imr” and “Mr” are matrices of size $N_x \times N_y \times N_c$. Note that SPIRiT replaces all data points in k-space with the estimated values. Do not enforce data consistency at the end of the “spirit” function, as we will do that separately later on.

Select a suitable regularization weight λ (by trial and error) when computing the SPIRiT kernel. Then, perform SPIRiT reconstruction for one iteration only. Display the magnitude images for all coils, and the corresponding k-space spectrums. Perform a SoS combination of coil images and display the final image using a window of `[0 max_val]`. Display the error images using a window of `[0 0.2*max_val]`. Compute PSNR and SSIM. Briefly comment on the results.

6) Perform one full iteration of SPIRiT reconstruction by enforcing data consistency to the output of “spirit” function that you got in Part (4). For data consistency, replace the reconstructed k-space data with the acquired data at sampled k-space locations. In MATLAB, `Mr(mask)=Mu(mask)` performs this action in one line. Use the same λ as in Part (5).

Display the magnitude images for all coils and the corresponding k-space spectrums. Then, perform a SoS combination of coil images and display the final image using a window of `[0 max_val]`. Display the error images using a window of `[0 0.2*max_val]`. Compute PSNR and SSIM. Briefly comment on the results.

7) Perform 10 full iterations of SPIRiT reconstruction (i.e., apply kernel in k-space followed by data consistency). Note that the kernel only needs to be computed once at the beginning and is then used for all iterations. Select a suitable regularization weight λ (by trial and error).

At the end of each iteration, perform a SoS combination of coil images, and compute PSNR and SSIM. Plot PSNR and SSIM as a function of iteration number. For the last iteration only, display the magnitude images for all coils and the corresponding k-space spectrums. Also, for the last iteration only, display the final SoS image using a window of $[0 \text{ } max_val]$ and display the error images using a window of $[0 \text{ } 0.2*max_val]$. Briefly comment on the results.

8) Iterative l_1 -SPIRiT Reconstruction: Now, we will apply SPIRiT and l_1 regularization iteratively, with an added data consistency step in between. Write a function `[imr, Mr] = llspirit(Mu, mask, kernel, beta, Niter)` that performs the steps listed below for “Niter” iterations. This function should return the resulting k-space data and the corresponding coil images from all iterations. So, both “imr” and “Mr” are matrices of size $N_x \times N_y \times N_c \times \text{Niter}$.

Before the iterations, compute the SPIRiT kernel once, using a suitable λ of your choice.

Here are the steps for each iteration:

1. Enforce data consistency in k-space by replacing the reconstructed k-space data with the acquired data at sampled k-space locations.
2. Apply a single iteration of SPIRiT (i.e., apply the kernel) to reconstruct k-space data for each coil and the corresponding images for each coil.
3. Apply l_1 regularization in wavelet domain to each coil image. β is kept constant across iterations.
4. Take centered 2D FFT for each coil image to compute the corresponding k-space data.

Here, you will use the functions you have written in Part (3) and (5), with the added data consistency step.

Choose a reasonable β (by trial and error). Perform one iteration of l_1 -SPIRiT. Display the magnitude images for all coils, and the corresponding k-space spectrums. Perform a SoS combination of coil images and display the final image using a window of $[0 \text{ } max_val]$. Display the error images using a window of $[0 \text{ } 0.2*max_val]$. Compute PSNR and SSIM. Briefly comment on the results.

9) Perform 10 iterations of l_1 -SPIRiT reconstruction. For each iteration, perform a SoS combination of coil images, and compute PSNR and SSIM. Plot PSNR and SSIM as a function of iteration number. For the last iteration only, display the final SoS image using a window of $[0 \text{ } max_val]$ and display the error images using a window of $[0 \text{ } 0.2*max_val]$. Briefly comment on the results.

Final Remark: A 3×3 kernel may be too small for SPIRiT reconstruction, especially in the outer k-space regions where we have fewer acquired data. Typically, 5×5 or 7×7 kernels may be preferred. Therefore, some of the results in this homework may fall short of demonstrating the true performance of SPIRiT.